

PRODUCT REQUIREMENTS DOCUMENT

AI-Powered ServiceNow ITSM
Regression Test Suite Management &
Intelligent Impact Analysis Platform

REGIQ Enterprise v1.0

Classification: Confidential | Version: 1.0.0 | Date: June 2025

Document Attribute	Details
Product Name	REGIQ — Intelligent Regression & Impact Analysis Platform
Product Version	1.0 (Initial Release)
Document Status	Draft for Review
Target Audience	Fortune 500 Enterprise ITSM Organizations
Primary Domain	ServiceNow ITSM — Regression Test Suite Management
Document Owner	Product Management — AI QA Platform Division
Stakeholders	QA Engineering, Release Management, DevOps, Business Teams
Review Cycle	Bi-weekly during Phase 1; Monthly post-launch

Table of Contents

- Table of Contents 2
- 1. Executive Summary 5
 - 1.1 Product Overview 5
 - 1.2 Enterprise QA Challenges Addressed 5
 - 1.3 AI-Powered Testing Vision 5
 - 1.4 Business Benefits and ROI Expectations 5
- 2. Problem Statement 7
 - 2.1 Complexity of Enterprise ServiceNow Test Repositories 7
 - 2.2 Manual Regression Suite Maintenance 7
 - 2.3 Difficulty Identifying Impacted Test Areas 7
 - 2.4 High Testing Effort and Slow Release Cycles..... 7
 - 2.5 Defect Leakage to Production 7
 - 2.6 Lack of Intelligent Prioritization and Cross-Module Dependency Complexity 7
- 3. Product Vision..... 8
 - 3.1 Autonomous Intelligent QA Operations..... 8
 - 3.2 AI-Driven Impact Analysis 8
 - 3.3 Smart Regression Optimization 8
 - 3.4 Predictive Release Quality Management..... 8
- 4. Product Goals and Key Performance Indicators..... 9
 - 4.1 Primary Product Goals 9
 - 4.2 AI Model-Specific KPIs 9
- 5. Scope 11
 - 5.1 In Scope — Phase 1–5..... 11
 - 5.2 Out of Scope — Phase 1 11
- 6. Stakeholders..... 12
- 7. User Personas 13
 - 7.1 Persona 1: Priya — Senior QA Engineer 13
 - 7.2 Persona 2: Marcus — Test Manager 13
 - 7.3 Persona 3: Aisha — ServiceNow Platform Administrator 13
 - 7.4 Persona 4: Raj — Release Manager 13
 - 7.5 Persona 5: Jordan — DevOps / CI-CD Engineer 14
- 8. Functional Requirements 15
 - 8.1 ServiceNow Integration Module..... 15
 - 8.1.1 Core Integration Capabilities 15
 - 8.1.2 Update Set Parsing Engine 15
 - 8.2 Enterprise Test Repository 16
 - 8.2.1 Repository Management Capabilities..... 16

- 8.3 Intelligent Classification Engine 17
 - 8.3.1 Business Process Classification..... 17
 - 8.3.2 Criticality Scoring Mechanism 17
- 8.4 Regression Test Suite Engine 18
- 8.5 Defect Intelligence Engine 18
- 8.6 Change Intelligence Engine..... 19
- 8.7 Intelligent Impact Analysis Engine 19
 - 8.7.1 Impact Analysis Workflow 19
 - 8.7.2 Confidence Scoring Model 20
- 8.8 Reporting and Dashboards..... 20
- 9. AI/ML Architecture 22
 - 9.1 Overview of AI Models..... 22
 - 9.2 NLP Pipeline Architecture..... 22
 - 9.3 Feature Engineering for ML Models..... 23
 - 9.4 Model Training and Feedback Loops..... 23
 - 9.5 Explainable AI (XAI) 24
- 10. System Architecture..... 25
 - 10.1 High-Level Architecture Overview..... 25
 - 10.2 Frontend Architecture 25
 - 10.3 Backend Services Architecture..... 26
 - 10.4 AI Services Architecture 26
- 11. Dependency Mapping Engine 28
 - 11.1 Overview and Architecture 28
 - 11.2 Graph Schema (Neo4j)..... 28
 - 11.3 Impact Traversal Algorithms 29
 - 11.3.1 Breadth-First Traversal for Downstream Impact..... 29
 - 11.3.2 Upstream Dependency Tracing..... 29
 - 11.3.3 Critical Path Analysis 29
- 12. Database Design 30
 - 12.1 Relational Schema (PostgreSQL)..... 30
 - 12.1.1 Core Entity Schemas 30
 - 12.2 Document Store Schema (MongoDB) 31
- 13. API Design..... 32
 - 13.1 API Standards and Authentication..... 32
 - 13.2 Core API Endpoints..... 32
 - 13.3 Impact Analysis API — Request/Response Example 33
- 14. UI/UX Requirements 34
 - 14.1 Design Principles..... 34
 - 14.2 Key Screen Descriptions 34

- 15. Security Requirements..... 36
- 16. DevOps and Deployment Architecture 37
 - 16.1 Containerization and Orchestration 37
 - 16.2 CI/CD Pipeline..... 37
 - 16.3 Observability Stack..... 37
- 17. Platform Testing Strategy..... 39
- 18. Risk Register and Mitigation Strategies..... 40
- 19. Future Enhancements Roadmap 42
- 20. Success Metrics Framework 43
- 21. Product Roadmap 44
- 22. Project Deliverables 45

1. Executive Summary

1.1 Product Overview

REGIQ (Regression Intelligence Quotient) is an enterprise-grade, AI-powered Software-as-a-Service (SaaS) platform purpose-built for Fortune 500 organizations operating on ServiceNow ITSM. It unifies intelligent regression test suite management, automated impact analysis, defect intelligence, and predictive release quality scoring into a single cohesive platform.

REGIQ eliminates the chronic inefficiency of manual regression suite maintenance by applying advanced machine learning, natural language processing, graph-based dependency analysis, and AI-assisted impact recommendations. The platform continuously ingests ServiceNow metadata—update sets, business rules, workflows, API changes, catalog items, and integrations—and translates this change intelligence into actionable, risk-ranked regression test suites in minutes rather than days.

1.2 Enterprise QA Challenges Addressed

Enterprise ServiceNow environments routinely host 10,000–50,000+ test cases spanning dozens of ITSM modules, hundreds of third-party integrations, and complex multi-process workflows. The absence of intelligent tooling creates cascading problems:

- Regression suites are built manually by tribal knowledge, leading to coverage gaps and over-testing
- Impact analysis requires days of cross-functional meetings with no data-driven foundation
- Defect leakage to production causes SLA breaches and customer trust erosion
- Release cycles extend by 30–60% due to undifferentiated full-suite regression runs
- Teams lack visibility into upstream/downstream test case dependencies

1.3 AI-Powered Testing Vision

REGIQ's AI vision is predicated on four pillars: Intelligent Classification (every test case is automatically classified by business process, type, criticality, and dependency), Autonomous Impact Detection (change events trigger real-time identification of impacted test cases with confidence scores), Predictive Defect Intelligence (historical defect patterns power proactive risk identification before releases), and Continuous Learning (every test execution, defect, and release outcome feeds back into the AI models to improve accuracy over time).

1.4 Business Benefits and ROI Expectations

Benefit Category	Expected Improvement	Measurement Method
Regression Suite Effort Reduction	55–70%	Man-hours per release cycle
Production Defect Leakage	40–60% reduction	Defects per 1,000 test cases post-release
Release Cycle Duration	30–45% faster	Lead time from code freeze to production
Impact Analysis Accuracy	>90% precision	True positive rate on impacted test identification
AI Classification Accuracy	>92%	Manual vs AI classification comparison
Test Automation Coverage Growth	+25–35%	Automated / Total test case ratio

Benefit Category	Expected Improvement	Measurement Method
Mean Time to Validation (MTTV)	60% improvement	Hours from change detection to regression completion
ROI Payback Period	8–14 months	Cost savings vs platform investment

2. Problem Statement

2.1 Complexity of Enterprise ServiceNow Test Repositories

Large enterprises deploying ServiceNow ITSM accumulate test repositories that routinely exceed 15,000–50,000 individual test cases. These repositories span core ITSM modules (Incident, Change, Problem, CMDB), service management extensions (HRSD, ITOM, SecOps), and complex third-party integrations (SAP, Workday, Microsoft Azure DevOps, Jira, PagerDuty). Managing this corpus manually is untenable at enterprise scale.

2.2 Manual Regression Suite Maintenance

Without intelligent tooling, regression suite maintenance relies entirely on individual QA engineer expertise. Suites become stale between releases, failing to reflect architectural changes, new integrations, or deprecated functionality. Maintenance overhead consumes 20–30% of total QA bandwidth in large organizations.

2.3 Difficulty Identifying Impacted Test Areas

When a ServiceNow update set modifies business rules, workflow activities, or API schemas, identifying the full blast radius of impacted test cases requires manual tracing across dependency chains. Without automated dependency graphs, critical regression gaps are frequently missed, leading to defect escapes that manifest in production.

2.4 High Testing Effort and Slow Release Cycles

The absence of intelligent test selection forces teams to run full regression suites for every release. This non-selective approach inflates testing cycles by 3–5x compared to optimized impact-based regression. Large enterprises report 4–8 week testing windows that delay business-critical deployments and accumulate technical debt.

2.5 Defect Leakage to Production

Industry benchmarks indicate that 15–25% of production defects in ServiceNow environments could have been detected through adequately scoped regression testing. Root causes include: unidentified dependency chains in impact analysis, test case mapping gaps in modified workflow areas, inadequate coverage of third-party integration regression points, and failure to re-test historically unstable modules.

2.6 Lack of Intelligent Prioritization and Cross-Module Dependency Complexity

ServiceNow's architecture creates deep cross-module dependencies. A single business rule modification can cascade through Incident routing logic, Change risk assessment, CMDB relationship queries, and Service Catalog fulfillment workflows simultaneously. Without a dependency graph engine, QA teams have no systematic method to identify and validate this blast radius.

3. Product Vision

REGIQ envisions a future state where enterprise QA operations in ServiceNow environments are predominantly autonomous. The platform's north star is zero-effort regression planning: where change detection, dependency analysis, test selection, suite optimization, and execution scheduling occur with minimal human intervention, guided by transparent AI recommendations that QA engineers trust and validate.

3.1 Autonomous Intelligent QA Operations

The platform progressively automates QA decision-making from descriptive (what changed) to diagnostic (what is impacted) to prescriptive (what should be tested) to predictive (what is likely to fail). This maturity progression aligns with enterprise AI adoption curves and builds stakeholder confidence at each stage.

3.2 AI-Driven Impact Analysis

At the core of the vision is an Impact Analysis Engine that operates in real time as update sets are promoted, requirements are raised, or configuration changes are committed. The engine combines graph traversal across dependency trees, vector similarity matching across test case embeddings, historical failure probability scoring, and business criticality weights to produce a ranked, confidence-scored list of test cases that must be included in the regression suite for a given change.

3.3 Smart Regression Optimization

REGIQ's Regression Suite Optimizer eliminates redundancy, detects overlapping coverage, balances execution time against risk coverage, and generates multiple execution strategies (Fast Track, Standard, Full Coverage) enabling release managers to make informed time-vs-risk trade-offs backed by data.

3.4 Predictive Release Quality Management

Beyond reactive testing, REGIQ provides predictive release quality scoring: a composite metric that factors historical defect density per module, change complexity score, coverage completeness, and team execution velocity to forecast the probability of production incidents before release sign-off.

4. Product Goals and Key Performance Indicators

4.1 Primary Product Goals

Goal ID	Product Goal	Target Metric	Measurement Frequency
G-01	Reduce regression test execution effort	55% reduction in man-hours per release	Per release cycle
G-02	Improve impact analysis accuracy	>90% precision, >88% recall on impacted test identification	Monthly AI model evaluation
G-03	Reduce production defect leakage	40% reduction in post-release production defects	Quarterly comparison
G-04	Increase AI classification accuracy	>92% accuracy vs manual classification baseline	Monthly validation audit
G-05	Accelerate release validation cycles	Reduce validation cycle duration by 35%	Per release
G-06	Predictive risk scoring accuracy	Risk score correlation >0.85 with actual defect outcomes	Per release retrospective
G-07	Increase automation coverage ratio	+25% automated test coverage growth per quarter	Quarterly
G-08	Repository completeness	100% test cases mapped to at least one business process and criticality level	Monthly audit
G-09	Improve release confidence	Release Readiness Score >85% before production deployments	Per release
G-10	Reduce mean time to validation (MTTV)	MTTV from change detection to regression completion <48 hours	Per release cycle

4.2 AI Model-Specific KPIs

AI Model	Primary KPI	Target	Monitoring Method
Test Case Classifier	Multi-label classification F1-score	>0.90	Holdout test set, monthly
Impact Analysis Engine	Precision / Recall at top-20 recommended tests	P>0.90 / R>0.88	Release retrospective labeling
Defect Recurrence Predictor	AUC-ROC for defect repeat prediction	>0.87	Historical defect comparison
Criticality Scoring Model	Rank correlation with business impact outcomes	>0.85 Spearman	Quarterly stakeholder review
Similarity Embedding Model	Mean Reciprocal Rank (MRR) for related test retrieval	>0.88	Monthly query analysis
Risk Score Model	Brier Score on release quality prediction	<0.15	Post-release retrospective

5. Scope

5.1 In Scope — Phase 1–5

Scope Area	Included Capabilities
ServiceNow Modules	Incident Management, Change Management, Problem Management, CMDB, Service Catalog, Asset Management, HRSD, ITOM, Security Operations (SecOps), Event Management, GRC
Test Repository Management	Centralized test case repository, versioning, bulk operations, tagging, search, filtering, import/export (Excel, CSV, ServiceNow native)
AI Classification Engine	Business process mapping, criticality scoring, test type classification, dependency classification, multi-label tagging
Impact Analysis Engine	Update set parsing, change detection, dependency graph traversal, impacted test identification, confidence scoring, risk-ranked suite generation
Defect Intelligence Engine	Historical defect analysis, repeat defect prediction, defect clustering, risk heatmaps, integration with Jira/ADO/ServiceNow defects
Regression Suite Management	Suite creation, optimization, execution strategy generation, coverage analysis, execution history
Dependency Mapping Engine	Graph-based dependency analysis, upstream/downstream tracing, Neo4j graph store, visual dependency maps
Reporting & Dashboards	Release readiness, impact analysis, defect trends, coverage heatmaps, AI recommendation panels, executive dashboards
Integrations	ServiceNow REST API, Jira, Azure DevOps, Jenkins, GitHub Actions, LDAP/SSO, Email/Slack notifications
Security & Compliance	RBAC, OAuth2/OIDC, JWT, AES-256 encryption, audit logging, SOC 2 Type II controls
Deployment	Docker containers, Kubernetes orchestration, multi-tenant SaaS, on-premise option

5.2 Out of Scope — Phase 1

- Test case execution engine (integration with existing tools: ATF, Selenium Grid)
- Automated defect filing and ticket creation (future Phase 4)
- ServiceNow non-ITSM modules: CSM, FSM, ITBM (planned Phase 3 extension)
- Self-healing test scripts and autonomous remediation (Phase 5)
- Direct ServiceNow instance modification or configuration (read-only integration)
- Mobile test case management application (planned Phase 3)

6. Stakeholders

Stakeholder Role	Primary Interest	Level of Influence	Communication Frequency
QA Engineers	Accurate impact analysis, reduced manual suite maintenance, clear test prioritization	High — daily users	Daily
Test Managers / QA Leads	Coverage metrics, team productivity, release confidence scores, AI recommendation quality	High — approval authority	Daily / Per Release
Product Owners / Business Analysts	Requirement-to-test traceability, feature release risk visibility, sprint QA readiness	Medium	Per Sprint
ServiceNow Administrators	Update set analysis accuracy, sync reliability, metadata extraction completeness	High — integration owners	Weekly
Application Developers	Change impact visibility, reduced back-and-forth on regression scope, defect pattern insights	Medium	Per Release
DevOps / CI-CD Engineers	Pipeline integration, automated suite triggering, execution orchestration	High — integration consumers	Per Deployment
Release Managers	Release readiness scoring, go/no-go dashboard, risk visibility, compressed timelines	High — decision makers	Per Release
IT Security Teams	ACL change impact, SecOps module regression coverage, audit trail completeness	Medium	Quarterly / Per Release
Business Stakeholders / CIO	ROI metrics, production defect trends, QA maturity benchmarks, SLA compliance	High — funding authority	Monthly / Quarterly
Platform Vendor (REGIQ)	Product roadmap execution, customer success, AI model performance, adoption metrics	Highest	Continuous

7. User Personas

7.1 Persona 1: Priya — Senior QA Engineer

Priya — Senior QA Engineer | ServiceNow ITSM | 7 Years Experience

Goals: Execute precise, impact-driven regression suites without wasting cycles on unaffected areas.

Trust AI recommendations with explainable reasoning.

Pain Points: Spends 40% of sprint time manually identifying impacted tests. No tooling to visualize upstream/downstream dependencies. Tribal knowledge drives suite construction.

Responsibilities: Test case authoring, regression suite execution, defect reporting, impact analysis coordination.

Workflow with REGIQ: Receives AI-generated impact report on release change set → reviews dependency-highlighted test cases → approves or adjusts suite → executes → reviews defect correlation feedback.

7.2 Persona 2: Marcus — Test Manager

Marcus — Test Manager | Global Financial Services Enterprise | 12 Years Experience

Goals: Demonstrate QA value through measurable KPIs. Optimize team bandwidth. Provide release confidence dashboards to stakeholders.

Pain Points: No single view of regression coverage across all ServiceNow modules. Manual reporting takes 2 days per release. Release managers demand faster sign-off.

Responsibilities: Suite strategy, team allocation, release readiness certification, stakeholder reporting.

Workflow with REGIQ: Monitors Release Readiness Dashboard → reviews AI risk scores per module → approves optimized suite strategy → signs off using confidence scoring reports.

7.3 Persona 3: Aisha — ServiceNow Platform Administrator

Aisha — ServiceNow Platform Administrator | Healthcare Enterprise | 9 Years Experience

Goals: Ensure update set changes are accurately reflected in regression impact analysis. Maintain sync integrity between ServiceNow instance and REGIQ.

Pain Points: Cannot easily communicate which configuration changes have QA implications. No automated notification to QA team when update sets are promoted.

Responsibilities: Update set management, system configuration, integration ownership, platform governance.

Workflow with REGIQ: Promotes update set → REGIQ auto-parses metadata → notifications dispatched to QA team with change impact summary → Aisha validates parsed changes.

7.4 Persona 4: Raj — Release Manager

Raj — Release Manager | Telecommunications Enterprise | 10 Years Experience

Goals: Compress release validation windows. Get objective go/no-go signals. Balance release speed with production risk.

Pain Points: Release readiness is a subjective judgment call. No data-driven risk scoring. Testing timelines frequently extend without advance warning.

Responsibilities: Release planning, go/no-go decisions, cross-team coordination, production deployment oversight.

Workflow with REGIQ: Reviews Release Readiness Score → analyzes uncovered risk areas → runs 'What If' simulation (skip X tests → risk increases by Y%) → makes informed go/no-go decision.

7.5 Persona 5: Jordan — DevOps / CI-CD Engineer

Jordan — DevOps Engineer | E-Commerce Enterprise | 6 Years Experience

Goals: Integrate REGIQ impact analysis into CI/CD pipelines. Trigger regression suites automatically on ServiceNow deployment events. Reduce manual QA hand-offs.

Pain Points: QA is a manual bottleneck in deployment pipelines. No API-first tooling for regression suite triggering. Lack of pipeline-native test impact data.

Responsibilities: CI/CD pipeline management, test automation integration, deployment orchestration.

Workflow with REGIQ: Jenkins pipeline triggers REGIQ API on deployment event → impact analysis returned as JSON → suite executed in parallel → results fed back to pipeline for pass/fail gate.

8. Functional Requirements

8.1 ServiceNow Integration Module

8.1.1 Core Integration Capabilities

Requirement ID	Requirement	Priority	Notes
FR-INT-001	REST API integration with ServiceNow instance using OAuth2 client credentials flow	Critical	Supports multiple instance connections
FR-INT-002	Incremental sync of test-relevant metadata (business rules, workflows, catalog items, ACLs)	Critical	Delta sync every 15 mins; full sync nightly
FR-INT-003	Update set parsing: extract changed components with type, scope, and modification details	Critical	Supports XML-based update set analysis
FR-INT-004	ServiceNow table-level metadata extraction: table schema, field definitions, relationships	High	Required for dependency graph construction
FR-INT-005	Support for multiple ServiceNow instances (DEV, TEST, UAT, PROD) per tenant	High	Environment-aware sync configuration
FR-INT-006	Webhook registration for real-time update set promotion events	High	Falls back to polling if webhook unavailable
FR-INT-007	ServiceNow ATF (Automated Test Framework) test case import	High	Map ATF tests to REGIQ repository
FR-INT-008	ServiceNow defect and incident import for defect intelligence engine	Medium	Filter by module, date range, severity
FR-INT-009	Connection health monitoring with automated re-authentication on token expiry	High	Alert on sync failures
FR-INT-010	Support for ServiceNow scoped applications and global scope test separation	High	Scope-aware classification

8.1.2 Update Set Parsing Engine

The Update Set Parser is a critical subsystem that transforms raw ServiceNow update set XML into structured change intelligence. The parser must identify and extract all change types relevant to regression impact analysis.

Component Type	Extracted Attributes	Dependency Mapping Impact
Business Rules	Name, table, when/order, script content, active status, conditions	High: affects table-level and process-level test cases
Workflow Activities	Workflow name, stage, activity type, conditions, transitions	High: cascades through all tests for related process
UI Policies / UI Actions	Form, field, condition, action type	Medium: affects UI test cases for related tables
ACL / Security Policies	Table, operation, condition, script	High: SecOps and CMDB test cases impacted

Component Type	Extracted Attributes	Dependency Mapping Impact
Script Includes	Name, script body, client callable flag	High: cross-cutting impact; dependency graph traversal required
REST API Definitions	Endpoint, method, schema changes	High: all inbound/outbound integration tests
Catalog Items / Variables	Item name, category, variable set, workflow mapping	Medium-High: Service Catalog test cases
Scheduled Jobs	Name, run schedule, script reference	Medium: batch process test cases
Email Notifications	Event, condition, template, recipients	Low-Medium: notification test cases
Table Definitions / Fields	Table name, new/modified fields, type changes	High: requires re-evaluation of all table-scoped tests

8.2 Enterprise Test Repository

8.2.1 Repository Management Capabilities

The Enterprise Test Repository is the single source of truth for all test cases across the organization's ServiceNow ITSM ecosystem. It supports the full test lifecycle from authoring through archival.

Requirement ID	Requirement	Priority
FR-REPO-001	Hierarchical repository structure: Tenant → Product → Module → Business Process → Test Suite → Test Case	Critical
FR-REPO-002	Full test case lifecycle management: Draft, Review, Approved, Active, Deprecated, Archived	Critical
FR-REPO-003	Multi-format test case authoring: structured fields, step-by-step format, Gherkin BDD syntax	High
FR-REPO-004	Version history and audit trail for all test case modifications with diff comparison	High
FR-REPO-005	Bulk import from Excel/CSV with field mapping wizard and validation preview	High
FR-REPO-006	Bulk export to Excel, PDF, and ServiceNow ATF format	High
FR-REPO-007	Multi-dimensional tagging: business process, criticality, type, integration, owner, release	Critical
FR-REPO-008	Advanced search with faceted filtering, full-text search, and saved search profiles	High
FR-REPO-009	Test case clone with relationship inheritance and auto-incremented ID	Medium
FR-REPO-010	Duplicate detection using NLP similarity scoring before test case creation	High
FR-REPO-011	Traceability matrix: test case ↔ requirement ↔ defect ↔ release linkage	High

Requirement ID	Requirement	Priority
FR-REPO-012	Test case reuse analytics: identify frequently duplicated logic for consolidation	Medium

8.3 Intelligent Classification Engine

8.3.1 Business Process Classification

Every test case in the repository must be classified against the organization's ServiceNow business process hierarchy. The Classification Engine automates this mapping using NLP analysis of test case text combined with rule-based category matching.

Classification Dimension	Sub-Categories	AI Model Used
Business Process	Incident Mgmt, Change Mgmt, Problem Mgmt, CMDB, Service Catalog, Asset Mgmt, HRSD, ITOM, SecOps, Event Mgmt, GRC	Fine-tuned BERT multi-class classifier
Criticality Level	Critical / High / Medium / Low — scored via composite business impact model	Rule engine + gradient boosted scoring model
Test Case Type	Inbound, Outbound, Reports, Processes, APIs, Integrations, UI, Workflow, Notifications, Scheduled Jobs, Batch	Multi-label text classifier
Dependency Class	Upstream, Downstream, Cross-module, Integration, Standalone	Graph-assisted classification + NLP
Automation Feasibility	Fully Automatable, Partially Automatable, Manual-only	Heuristic rule engine + ML scorer
Execution Frequency	Every Release, Monthly, Quarterly, On-demand, Smoke only	Historical execution pattern analysis

8.3.2 Criticality Scoring Mechanism

The Criticality Scoring Engine assigns a weighted composite score to each test case based on the following factors:

Factor	Weight	Scoring Criteria	Score Range
Revenue Impact	30%	Does failure impact revenue-generating processes or SLA penalties?	0–10
Business Process Criticality	25%	Is the process mission-critical (Incident Mgmt, Change) vs supporting?	0–10
Production Risk	20%	Historical correlation with production incidents when skipped	0–10
User / Customer Impact	15%	External-facing vs internal-only functionality	0–10
Security Sensitivity	10%	Involves ACLs, data access, SecOps workflows	0–10
COMPOSITE SCORE	100%	Weighted average; Critical ≥8.5, High 6.5–8.4, Medium 4.0–6.4, Low <4.0	0–10

8.4 Regression Test Suite Engine

The Regression Suite Engine is responsible for generating, optimizing, and managing regression test suites based on release scope, risk scoring, available execution time, and AI recommendations.

Suite Strategy	Description	Use Case	Typical Coverage
Fast Track Suite	Critical and High test cases in impacted modules only. Estimated execution ≤4 hours.	Emergency hotfixes, minor configuration changes	20–30% of full suite
Standard Suite	All impacted test cases with risk score >5.0. Balances coverage with velocity.	Regular sprint releases, minor enhancements	45–65% of full suite
Full Coverage Suite	All impacted plus historically unstable modules. Maximum defect detection.	Major releases, platform upgrades	80–100% of full suite
Smoke Suite	Sanity checks only — top 50 Critical test cases across all processes.	Post-deployment verification, environment validation	5–10% of full suite
Integration Suite	All inbound/outbound integration and API test cases. Focused on integration stability.	Third-party integration changes, API version upgrades	Integration test cases only
AI-Optimized Suite	ML-recommended test cases based on current change context and historical failure patterns.	AI-driven precision testing with confidence scoring	Variable — AI-determined

8.5 Defect Intelligence Engine

The Defect Intelligence Engine aggregates, analyzes, and learns from historical defect data to improve regression prioritization and predict future failure areas. It integrates with external defect management tools and ServiceNow's native incident/problem management.

Capability	Description	Data Sources
Defect Ingestion	Automated import of defects from Jira, Azure DevOps, ServiceNow Incidents. Normalized schema with severity, component, release, resolution.	Jira API, ADO API, ServiceNow REST
Repeat Defect Detection	Identifies defects that recur across multiple releases in the same module or test case area using NLP clustering and exact-match analysis.	REGIQ defect history database
Failure Pattern Analysis	Statistical analysis of failure frequency per test case, module, and release type. Generates module stability scores.	Execution history + defect links
Defect Clustering	Groups semantically similar defects using sentence embedding models (SBERT) and DBSCAN clustering. Identifies defect themes.	Defect descriptions + stack traces
Risk Heatmap Generation	Visualizes defect density per ServiceNow module × time period matrix. Highlights chronically unstable areas.	Aggregated defect history

Capability	Description	Data Sources
Regression Prioritization Input	Defect-weighted priority scores feed into the Impact Analysis Engine to elevate test cases in historically defect-prone areas.	Scoring model feature input

8.6 Change Intelligence Engine

The Change Intelligence Engine is the sensor layer of REGIQ. It continuously monitors ServiceNow environments for configuration changes, requirement updates, and enhancement deployments, translating raw change data into structured impact intelligence.

Change Type	Detection Method	Analysis Output
Update Set Promotion	Webhook event + polling fallback on ServiceNow sys_update_set table	Component list, change type, scope, affected tables, estimated impact radius
Business Rule Modification	Script delta analysis using AST parsing, condition comparison	Affected table, execution context, downstream workflow impact
Workflow Change	Workflow XML diff analysis, activity comparison, transition logic changes	Process mapping update, impacted test cases by workflow stage
API Schema Change	OpenAPI/REST definition comparison, field addition/removal/type change	All inbound/outbound integration tests for affected endpoints
ACL Policy Change	Security policy diff, role-condition comparison	SecOps and data access test cases; risk elevation flag
Catalog Item Change	Variable set diff, workflow mapping, category reassignment	Service Catalog and fulfillment test cases
ServiceNow Platform Upgrade	Version metadata change detection, patch notes NLP parsing	Full suite risk elevation; upgrade-specific test activation
Third-Party Integration Change	External system webhook, API version change notification	All integration test cases for affected integration point

8.7 Intelligent Impact Analysis Engine

8.7.1 Impact Analysis Workflow

The Intelligent Impact Analysis Engine is the most technically complex subsystem of REGIQ. It orchestrates multiple AI models, graph algorithms, and scoring functions to produce an authoritative, confidence-scored list of test cases required for a given change set.

Phase	Process Step	AI/Algorithm	Output
Phase 1: Change Ingestion	Parse update set XML; extract component list with metadata	Rule-based parser + NLP entity extraction	Structured change manifest
Phase 2: Component Classification	Classify each changed component by type, scope, and test relevance	Fine-tuned classification model	Labeled component inventory

Phase	Process Step	AI/Algorithm	Output
Phase 3: Direct Mapping	Direct test-case-to-component matching via stored relationships	Graph lookup (Neo4j traversal)	Tier-1 directly impacted tests
Phase 4: Dependency Traversal	Traverse upstream/downstream dependency graph for indirect impacts	BFS/DFS graph traversal with depth limit	Tier-2 indirectly impacted tests
Phase 5: Similarity Matching	Find semantically related test cases not captured by direct/graph methods	SBERT vector similarity (cosine ≥ 0.78)	Tier-3 similarity-matched tests
Phase 6: Historical Weighting	Apply defect frequency scores and historical execution failure rates	Gradient boosted scoring model	Risk-weighted test list
Phase 7: Business Criticality Overlay	Multiply risk score by criticality weight to produce final priority rank	Weighted scoring formula	Priority-ranked test suite recommendation
Phase 8: Confidence Scoring	Generate per-test confidence score with source attribution	Ensemble confidence model	Confidence-annotated recommendations
Phase 9: Suite Optimization	Remove duplicates, resolve coverage redundancy, generate execution strategies	Greedy set cover + constraint optimization	Optimized suite options

8.7.2 Confidence Scoring Model

Every test case recommendation produced by the Impact Analysis Engine carries a Confidence Score (0.00–1.00) that indicates the system's certainty that the test case is genuinely impacted by the current change set. Confidence scores are calculated as a weighted ensemble of four source signals:

Signal	Weight	Description
Direct Dependency Match	40%	Test case directly linked to a modified component in the dependency graph
Graph Traversal Depth	25%	Score decays with traversal depth: depth-1 = 1.0, depth-2 = 0.75, depth-3 = 0.50, depth-4+ = 0.25
Semantic Similarity Score	20%	Cosine similarity between change description embedding and test case embedding
Historical Co-failure Rate	15%	Frequency at which this test case failed in past releases with similar change profiles
FINAL CONFIDENCE	100%	Displayed as: High (≥ 0.85), Medium (0.65–0.84), Low (0.45–0.64), Speculative (< 0.45)

8.8 Reporting and Dashboards

Dashboard	Primary Audience	Key Metrics Displayed	Refresh Rate
Release Readiness Dashboard	Release Managers, Test Managers	Readiness score, uncovered risk areas, suite completion %,	Real-time

Dashboard	Primary Audience	Key Metrics Displayed	Refresh Rate
		pass/fail rate, go/no-go recommendation	
Impact Analysis Dashboard	QA Engineers, Test Managers	Change manifest, impacted test count by tier, confidence distribution, dependency graph preview	Per analysis run
Regression Coverage Dashboard	Test Managers, QA Leads	Coverage % by module, test type distribution, unmapped test cases, criticality breakdown	Daily
Defect Intelligence Dashboard	QA Engineers, Test Managers	Defect heatmap by module, repeat defect rate, top unstable areas, defect trend over releases	Daily / Per Release
AI Recommendations Panel	QA Engineers	Top-ranked recommended test cases, confidence scores, source attribution, accept/reject workflow	Per Impact Analysis
Dependency Graph Explorer	QA Engineers, ServiceNow Admins	Interactive neo4j-powered graph, filter by module/type, path highlighting, impact radius visualization	Real-time
Executive QA Metrics Dashboard	CIO, QA Director, Business Stakeholders	MTTV trend, defect leakage rate, AI accuracy KPIs, ROI metrics, release confidence trend	Weekly / Monthly
Test Repository Health Dashboard	Test Managers, QA Leads	Repository completeness, stale test case count, unclassified tests, duplication index	Daily

9. AI/ML Architecture

9.1 Overview of AI Models

REGIQ's AI architecture consists of seven specialized models operating in concert across the classification, impact analysis, defect intelligence, and predictive quality domains. All models are trained on enterprise ITSM datasets and continuously refined through structured feedback loops.

Model Name	Type	Framework	Purpose
Test Case Classifier (TCC)	Multi-label text classification	Fine-tuned BERT (HuggingFace Transformers)	Classify test cases by business process, type, and dependencies
Impact Analysis Ranker (IAR)	Learning-to-Rank (LTR)	LightGBM + FAISS vector index	Rank impacted test cases by relevance and risk for a change set
Semantic Embedding Model (SEM)	Sentence embedding / dense retrieval	SBERT (all-mpnet-base-v2 fine-tuned)	Embed test cases and changes for semantic similarity matching
Defect Recurrence Predictor (DRP)	Binary + multi-class classification	XGBoost with temporal features	Predict probability of defect recurrence per module per release
Risk Score Model (RSM)	Regression + ordinal classification	Gradient Boosting Regressor (scikit-learn)	Compute composite release risk score and module risk scores
Dependency Graph Reasoner (DGR)	Graph Neural Network (GNN)	PyTorch Geometric (GraphSAGE)	Learn dependency propagation patterns across ServiceNow component graph
NLP Change Analyzer (NCA)	Named entity recognition + relation extraction	SpaCy + LangChain with GPT-4 augmentation	Parse update set descriptions and requirements for structured change intent

9.2 NLP Pipeline Architecture

The NLP pipeline processes three primary input streams: test case text (title, description, steps), change/requirement descriptions (update set names, release notes, user stories), and defect descriptions (summary, steps to reproduce, resolution notes).

Pipeline Stage	Process	Technology	Output
Text Ingestion	Normalize encoding, strip HTML/XML, handle ServiceNow-specific markup	Python BeautifulSoup + custom regex	Clean text corpus
Tokenization & Preprocessing	Tokenize, lowercase, remove stopwords, expand ITSM abbreviations	SpaCy tokenizer + custom ITSM vocabulary	Normalized token sequences

Pipeline Stage	Process	Technology	Output
Entity Recognition	Identify ServiceNow entities: table names, module references, API endpoints, role names	Custom SpaCy NER model trained on ITSM corpus	Structured entity annotations
Embedding Generation	Generate dense vector representations for test cases and changes	SBERT fine-tuned on ITSM QA pairs	512-dim dense vectors stored in Pinecone/pgvector
Intent Classification	Classify change intent: new feature, bug fix, configuration change, performance improvement	Fine-tuned BERT classifier	Change type label + confidence
Relation Extraction	Extract relationships between entities (e.g., 'workflow X triggers table Y')	Dependency parsing + relation extraction model	Structured relation triples
Summary Generation	Generate human-readable impact summaries for QA engineers	LangChain + GPT-4 (with prompt engineering)	Natural language impact explanation

9.3 Feature Engineering for ML Models

Feature Category	Features	Models Using
Test Case Text Features	TF-IDF unigrams/bigrams, SBERT embeddings, title/step length, keyword overlap with changed components	TCC, IAR, SEM
Change Context Features	Change type, component count, scope (global/scoped), update set size, release type, days since last change	IAR, RSM, DRP
Historical Execution Features	Pass rate (last 5 releases), failure streak count, last execution date, execution frequency, flakiness score	IAR, DRP, RSM
Dependency Graph Features	Node degree, betweenness centrality, graph depth, number of upstream/downstream dependencies, component type	DGR, IAR
Business Context Features	Process criticality score, module stability score, SLA sensitivity flag, integration dependency count, customer-facing flag	RSM, IAR, DRP
Defect History Features	Defect count per module (last 3 releases), repeat defect flag, MTTR, severity distribution, resolution category	DRP, IAR, RSM

9.4 Model Training and Feedback Loops

REGIQ implements a closed-loop learning architecture where every QA engineer interaction with AI recommendations generates feedback data that improves future model performance.

Feedback Event	Signal Type	Model Updated	Retraining Trigger
QA Engineer accepts AI-recommended test case	Positive reinforcement signal	Impact Analysis Ranker	Batch retraining weekly
QA Engineer rejects AI-recommended test case	Negative signal with optional reason code	Impact Analysis Ranker, SEM	Batch retraining weekly
Manual test case added post-impact analysis	Coverage gap signal	Dependency Graph Reasoner	Nightly incremental update
Test case that was not recommended fails in execution	False negative signal — critical	IAR, SEM, DGR	Triggered immediately; priority retraining
Test case recommended but passes across 5+ consecutive releases	Potential over-recommendation	RSM, IAR	Monthly review batch
New defect filed against a module	Risk elevation signal	Defect Recurrence Predictor	Real-time score update
Release quality post-deployment review	Release-level ground truth	Risk Score Model	Post-release batch update

9.5 Explainable AI (XAI)

Enterprise QA teams require transparent AI recommendations they can audit, challenge, and override. REGIQ implements Explainable AI across all decision-making models through the following mechanisms:

- **SHAP (SHapley Additive exPlanations):** Applied to gradient boosted models (IAR, RSM, DRP) to generate per-prediction feature importance explanations displayed inline in the UI
- **Source Attribution Cards:** Every recommended test case displays its recommendation sources (Direct Dependency, Graph Traversal Depth-N, Semantic Similarity Score X, Historical Failure Rate Y%)
- **Confidence Interval Display:** Models provide prediction intervals, not just point estimates, reflecting genuine model uncertainty
- **Counterfactual Explanations:** 'Why was this test case NOT recommended?' explanations available on request, showing which signals fell below threshold
- **Audit Trail API:** All AI decisions are logged with full input features, model version, and output scores accessible via API for compliance purposes

10. System Architecture

10.1 High-Level Architecture Overview

REGIQ adopts a microservices architecture organized around bounded service domains. Services communicate via asynchronous event streaming (Apache Kafka) for high-throughput data pipelines and synchronous REST/gRPC for request-response workflows. The architecture is designed for multi-tenant enterprise SaaS deployment on Kubernetes with optional on-premise deployment for regulated industries.

Service Domain	Services	Technology Stack	Scaling Strategy
Frontend Layer	Web Application, Dashboard Renderer, Dependency Graph Visualizer	Next.js 14, React 18, Tailwind CSS, D3.js, React-Flow	CDN-served static assets; serverless SSR
API Gateway Layer	Request routing, rate limiting, auth validation, tenant isolation	Kong API Gateway / AWS API Gateway	Horizontal scaling with load balancer
Core Backend Services	Test Repository Service, Suite Management Service, User Management Service	Node.js 20, Express.js, TypeScript	Horizontal pod autoscaling (HPA)
Integration Services	ServiceNow Connector, Jira Connector, ADO Connector, Webhook Manager	Node.js, Python FastAPI	Dedicated pods per integration; queue-based
AI Services Layer	Classification Service, Impact Analysis Service, Defect Intelligence Service, NLP Pipeline Service	Python 3.11, FastAPI, scikit-learn, PyTorch, LangChain	GPU-enabled pods for inference; CPU for batch
Graph Engine Service	Dependency Graph Builder, Impact Traversal Engine, Graph Query Service	Python, Neo4j Driver, NetworkX	Neo4j cluster with read replicas
Event Bus	Change events, sync events, analysis triggers, notification events	Apache Kafka 3.x	Kafka cluster with topic partitioning
Data Layer	Relational data, document data, vector store, graph store, cache	PostgreSQL 15, MongoDB 7, pgvector/Pinecone, Neo4j 5, Redis 7	Multi-region replication; automated failover

10.2 Frontend Architecture

The REGIQ frontend is a Next.js 14 application using the App Router paradigm with server-side rendering for dashboard pages and client-side interactivity for the dependency graph explorer and real-time panels.

- Component Library: Shadcn/UI + custom enterprise design system built on Tailwind CSS v3
- State Management: Zustand for global state; React Query v5 for server state and cache management
- Dependency Graph Visualization: React-Flow with custom Neo4j-backed node/edge rendering; D3.js for risk heatmaps and coverage charts

- Real-Time Updates: WebSocket connections via Socket.io for live impact analysis progress and execution monitoring
- Authentication: Next-Auth with OAuth2/OIDC support for enterprise SSO (Okta, Azure AD, ADFS)
- Accessibility: WCAG 2.1 AA compliant; keyboard navigation; screen reader support

10.3 Backend Services Architecture

Core backend services are implemented as TypeScript/Node.js Express services following REST API standards. Each service owns its data domain and communicates with other services through Kafka events or direct API calls for synchronous workflows.

Service	Responsibility	Database	Key APIs
Test Repository Service	CRUD for test cases, suites, classifications, versioning	PostgreSQL	POST /tests, GET /tests/{id}, PUT /tests/{id}/classify, GET /tests/search
Suite Management Service	Regression suite creation, optimization, execution tracking	PostgreSQL + MongoDB	POST /suites/generate, GET /suites/{id}/optimize, POST /suites/{id}/execute
Impact Analysis Service	Orchestrates impact analysis pipeline; calls AI services	PostgreSQL + Redis (cache)	POST /analysis/impact, GET /analysis/{id}/results, POST /analysis/{id}/feedback
Change Intelligence Service	Update set processing, change manifest generation, event emission	PostgreSQL + MongoDB	POST /changes/parse, GET /changes/{id}/manifest, GET /changes/history
Defect Intelligence Service	Defect ingestion, analytics, prediction requests	PostgreSQL + MongoDB	POST /defects/import, GET /defects/heatmap, GET /defects/predictions/{module}
Integration Service	Manages external system connections, credentials, sync schedules	PostgreSQL	POST /integrations/{type}/connect, POST /integrations/{id}/sync, GET /integrations/health
Notification Service	Dispatches email/Slack/webhook notifications on key events	PostgreSQL (log)	Event-driven via Kafka; POST /notifications/test
Auth & User Service	User management, RBAC, API key management, audit logging	PostgreSQL	POST /auth/login, GET /users/{id}/permissions, GET /audit/logs

10.4 AI Services Architecture

AI services are deployed as Python FastAPI applications with asynchronous request handling. Compute-intensive inference tasks (embedding generation, GNN inference) run on GPU-enabled Kubernetes pods with auto-scaling based on queue depth.

- Classification Service: Exposes /classify endpoint accepting test case text and returning multi-label classifications with confidence scores
- Impact Analysis AI Service: Accepts change manifest and test case embeddings; runs IAR model inference; returns ranked test case list with SHAP explanations
- Embedding Service: Batch and online embedding generation using SBERT; stores vectors in pgvector (PostgreSQL extension) or Pinecone for ANN search

- Defect Prediction Service: Accepts module name and release context; returns defect recurrence probability scores per module
- NLP Analysis Service: Accepts raw change text; runs NER, intent classification, relation extraction; returns structured change intelligence
- Model Registry: MLflow-based model versioning, staging, and production deployment tracking with A/B testing support

11. Dependency Mapping Engine

11.1 Overview and Architecture

The Dependency Mapping Engine (DME) is the structural foundation of REGIQ's impact analysis capability. It constructs and maintains a multi-dimensional dependency graph of the enterprise ServiceNow ecosystem, enabling precise identification of upstream and downstream impacts for any configuration change.

The DME is powered by Neo4j, a native graph database optimized for relationship traversal queries. The graph model captures five categories of relationships:

Relationship Category	Example Relationships	Graph Edge Type	Impact Weight
Direct Component Dependency	Business Rule → Table; Script Include → Business Rule	DEPENDS_ON	1.0 (full weight)
Workflow Dependencies	Workflow → Activity; Activity → Script Include; Workflow → Notification	TRIGGERS / EXECUTES	0.9
Integration Dependencies	REST API → Table; Integration → Workflow; External System → REST API	INTEGRATES_WITH	0.85
Process Dependencies	Service Catalog Item → Workflow; Incident → Assignment Rule → Group	FULFILLS / ROUTES_TO	0.75
Data Dependencies	Table → Table (via reference field); Table → CMDB Class	REFERENCES	0.65
Test Case Dependencies	Test Case → Component; Test Suite → Test Case; Test Case → Business Process	TESTS / COVERS / VALIDATES	N/A (meta-layer)

11.2 Graph Schema (Neo4j)

The Neo4j graph schema defines node types and relationship types that represent the complete topology of a ServiceNow instance's configuration and its test coverage layer.

Node Label	Key Properties	Relationships to Other Nodes
(:Component)	id, name, type, scope, table, script_hash, last_modified, active	DEPENDS_ON, TRIGGERS, EXECUTES, REFERENCES
(:BusinessProcess)	id, name, module, criticality, owner, sla_impact	HAS_COMPONENT, COVERS_TABLE, DEPENDS_ON
(:Table)	name, scope, parent_table, field_count, is_audited	REFERENCED_BY, EXTENDED_BY, HAS_RULE
(:Workflow)	id, name, table, version, active, stage_count	HAS_ACTIVITY, TRIGGERED_BY, CALLS_SCRIPT
(:Integration)	id, name, type, direction (in/out/both), protocol, target_system	USES_API, CALLS_WORKFLOW, MAPS_TABLE
(:TestCase)	id, title, criticality, type, status, automation_flag, embedding_id	TESTS, COVERS, LINKED_TO_DEFECT

Node Label	Key Properties	Relationships to Other Nodes
(:Defect)	id, title, severity, module, release, status, recurrence_count	LINKED_TO_COMPONENT, FOUND_IN_RELEASE
(:Release)	id, name, date, type, risk_score, environment	INCLUDES_CHANGE, EXECUTED_SUITE, HAD_DEFECT

11.3 Impact Traversal Algorithms

11.3.1 Breadth-First Traversal for Downstream Impact

When a component is modified (e.g., a Script Include), REGIQ performs a breadth-first traversal starting from the modified node and following outbound `DEPENDS_ON` and `TRIGGERS` relationships to identify all potentially impacted downstream components. The traversal applies depth-based weight decay to modulate confidence scores.

The traversal uses a maximum depth of 4 hops as default (configurable per tenant) to prevent unbounded graph walks in highly interconnected ServiceNow instances. A visited-node set prevents re-traversal of already-visited nodes, ensuring $O(V+E)$ complexity.

11.3.2 Upstream Dependency Tracing

For understanding what dependencies a changed component relies upon (upstream analysis), the DME performs a reverse traversal following inbound edges. This identifies components that the modified element depends on, which may need verification testing even if not directly changed.

11.3.3 Critical Path Analysis

For business-critical change sets, REGIQ computes critical paths through the dependency graph using a weighted shortest-path algorithm. Paths through high-criticality business processes receive elevated risk scores, ensuring that test cases covering critical paths are always included in regression suites regardless of depth.

12. Database Design

12.1 Relational Schema (PostgreSQL)

12.1.1 Core Entity Schemas

Table Name	Primary Key	Key Columns	Foreign Keys
test_cases	id (UUID)	title, description, status, criticality, business_process_id, type_tags[], automation_flag, embedding_id, created_by, version, parent_id	business_processes(id), users(id)
regression_suites	id (UUID)	name, release_id, strategy_type, status, total_tests, risk_score, coverage_pct, generated_by (user/ai), created_at	releases(id), users(id)
suite_test_cases	id (UUID)	suite_id, test_case_id, inclusion_reason, confidence_score, priority_rank, execution_status, result	regression_suites(id), test_cases(id)
releases	id (UUID)	name, version, environment, planned_date, status, risk_score, readiness_score, go_nogo_status	tenants(id)
change_sets	id (UUID)	release_id, source_type, update_set_ref, component_count, analysis_status, impact_count, created_at	releases(id)
change_components	id (UUID)	change_set_id, component_type, component_name, scope, table_name, change_type, raw_diff	change_sets(id)
defects	id (UUID)	external_id, source_system, title, severity, module, component_id, release_id, status, recurrence_count, is_repeat	change_components(id), releases(id)
execution_history	id (UUID)	test_case_id, suite_id, release_id, result, duration_ms, executed_by, executed_at, defects_found[]	test_cases(id), regression_suites(id), releases(id)
business_processes	id (UUID)	name, module, parent_id, criticality_level, sla_impact, revenue_impact, description	business_processes(id) (self-ref)
dependency_mappings	id (UUID)	source_type, source_id, target_type, target_id, relationship_type, weight, source_system, last_synced	—

Table Name	Primary Key	Key Columns	Foreign Keys
risk_scores	id (UUID)	entity_type, entity_id, release_id, score_value, score_version, factors_json, computed_at	releases(id)
ai_recommendations	id (UUID)	analysis_run_id, test_case_id, recommendation_rank, confidence_score, source_signals_json, accepted_by, accepted_at, rejected_reason	test_cases(id)

12.2 Document Store Schema (MongoDB)

Collection	Document Structure	Indexing Strategy
test_case_versions	{test_case_id, version, snapshot, changed_fields[], changed_by, timestamp}	Compound index: test_case_id + version
update_set_raw	{change_set_id, raw_xml, parsed_components[], metadata, tenant_id, sync_timestamp}	Index: tenant_id + sync_timestamp
ai_analysis_runs	{run_id, change_set_id, model_versions{}, input_features{}, outputs{ranked_tests[]}, runtime_ms, status}	Index: run_id; TTL index: 90 days
defect_clusters	{cluster_id, algorithm, defects[], theme_label, centroid_embedding, computed_at}	Index: cluster_id; geo-text on theme_label
notification_logs	{event_type, recipients[], payload, channel, status, sent_at, error_detail}	TTL index: 30 days

13. API Design

13.1 API Standards and Authentication

All REGIQ APIs follow RESTful conventions over HTTPS with JSON payloads. Authentication uses Bearer tokens (JWT) with a 1-hour expiry and refresh token rotation. API keys with HMAC-SHA256 request signing are available for machine-to-machine integration (CI/CD pipelines). All API requests are rate-limited per tenant and per user with standard HTTP 429 responses.

13.2 Core API Endpoints

Method	Endpoint	Description	Auth Scope
POST	/api/v1/analysis/impact	Trigger impact analysis for a change set. Returns analysis run ID.	qa:write
GET	/api/v1/analysis/{runId}/results	Retrieve ranked test case recommendations with confidence scores.	qa:read
POST	/api/v1/analysis/{runId}/feedback	Submit accept/reject feedback on AI recommendations.	qa:write
GET	/api/v1/tests	List test cases with pagination, filtering, and search.	qa:read
POST	/api/v1/tests	Create a new test case with auto-classification trigger.	qa:write
PUT	/api/v1/tests/{id}/classify	Re-classify a test case (manual override or AI re-run).	qa:write
POST	/api/v1/suites/generate	Generate optimized regression suite for a release.	qa:write
GET	/api/v1/suites/{id}/optimize	Get optimization options: Fast Track, Standard, Full Coverage.	qa:read
GET	/api/v1/dependencies/graph	Retrieve dependency graph nodes/edges for visualization.	admin:read
POST	/api/v1/changes/parse	Submit update set XML for parsing and change manifest generation.	admin:write
GET	/api/v1/defects/heatmap	Retrieve defect density heatmap data by module and release.	qa:read
GET	/api/v1/releases/{id}/readiness	Get release readiness score with contributing factor breakdown.	qa:read

Method	Endpoint	Description	Auth Scope
POST	/api/v1/integrations/servicenow/sync	Trigger manual sync from ServiceNow instance.	admin:write
GET	/api/v1/reports/coverage	Get regression coverage report in JSON or trigger Excel export.	qa:read

13.3 Impact Analysis API — Request/Response Example

POST /api/v1/analysis/impact — Request Body

```
{
  "release_id": "rel-2025-q2-001",
  "change_set_id": "cs-a7f3b291",
  "analysis_config": {
    "max_depth": 3,
    "min_confidence": 0.45,
    "strategy": "ai_optimized",
    "include_historical_weight": true,
    "criticality_filter": ["Critical", "High"]
  }
}
```

POST /api/v1/analysis/impact — Response (202 Accepted)

```
{
  "run_id": "run-c9d2e4f1-2025",
  "status": "processing",
  "estimated_completion_seconds": 45,
  "poll_url": "/api/v1/analysis/run-c9d2e4f1-2025/results",
  "webhook_url": "https://regiq.yourorg.com/webhooks/analysis/run-c9d2e4f1-2025"
}
```

14. UI/UX Requirements

14.1 Design Principles

- Enterprise Clarity: Information density balanced with cognitive clarity. Complex AI outputs presented in plain language with progressive disclosure for technical details.
- Action-Oriented: Every dashboard leads with actionable insights, not raw data. Primary CTAs (Accept Suite, View Impact, Run Analysis) are always visible.
- Trust through Transparency: AI confidence scores, source attributions, and explainability panels build engineer trust in automated recommendations.
- Responsive Performance: Dashboard renders <1.5s for standard views; dependency graph loads within 3s for graphs up to 10,000 nodes.

14.2 Key Screen Descriptions

Screen / View	Key UI Elements	Primary User Actions
Impact Analysis Dashboard	Change manifest accordion, tiered test case list (Tier-1/2/3 with color coding), confidence score bars, source attribution chips, accept/customize/export CTA panel	Review impact, accept suite, adjust scope, export for execution
Dependency Graph Explorer	React-Flow canvas with zoom/pan, node type legend, filter sidebar (by module, type, depth), path highlighting on hover, impact radius heat overlay	Explore dependencies, trace paths, filter by component type, export graph
Release Readiness Dashboard	Readiness score gauge (0–100), module-level readiness grid, risk area alerts, test execution progress bar, go/no-go recommendation panel	Monitor readiness, drill into risk areas, approve go/no-go
Test Repository Browser	DataGrid with faceted filters, multi-tag sidebar, quick preview panel, bulk action toolbar, duplicate detection banner	Search/filter tests, bulk classify, import/export
Regression Cockpit	Suite strategy selector, coverage donut chart, timeline estimates, risk score comparison across strategies, estimated defect detection probability	Select strategy, customize suite, schedule execution
Defect Intelligence Dashboard	Module heatmap (color intensity = defect density), repeat defect timeline chart, top-10 unstable modules table, prediction confidence bars	Identify risk areas, drill into defect history, export report
AI Recommendations Panel	Card-based recommendations with confidence indicators, SHAP waterfall chart in expandable section, accept/reject toggle, reason-code selector on reject	Accept/reject recommendations, provide feedback, view explanations
Executive Dashboard	KPI scorecard tiles (MTTV, leakage rate, coverage %), trend sparklines, AI accuracy trend, release timeline heatmap	Monitor QA program health, export executive report

15. Security Requirements

Security Domain	Requirement	Implementation
Authentication	OAuth2/OIDC for SSO; JWT for API access; MFA enforcement for admin roles	NextAuth.js + OAuth2 provider (Okta/Azure AD/ADFS); JWT RS256 signing
Authorization	Role-Based Access Control (RBAC) with 6 predefined roles: Super Admin, Tenant Admin, Test Manager, QA Engineer, Viewer, API Service Account	Middleware-enforced permission checks; database-row-level security for multi-tenant isolation
Data Encryption at Rest	AES-256 encryption for sensitive test data, credentials, API keys, and AI model outputs	PostgreSQL Transparent Data Encryption; MongoDB encrypted storage; KMS-managed keys (AWS KMS / Azure Key Vault)
Data Encryption in Transit	TLS 1.3 minimum for all internal and external communications	Enforced via API Gateway and service mesh (Istio); HSTS headers on frontend
API Security	HMAC-SHA256 request signing for CI/CD integrations; API key rotation; rate limiting per tenant	Kong API Gateway policies; Redis-based rate limiting
Audit Logging	All user actions, AI decisions, data access, and admin changes logged with immutable audit trail	PostgreSQL audit_logs table; forwarded to SIEM (Splunk/Elasticsearch)
Secrets Management	Zero plaintext secrets in code or config files; all credentials via secrets manager	Kubernetes Secrets + HashiCorp Vault integration
Vulnerability Management	Dependency scanning (Snyk), container image scanning (Trivy), SAST (SonarQube), DAST in CI pipeline	Integrated into GitHub Actions CI/CD pipeline; block on critical CVEs
Compliance	SOC 2 Type II, ISO 27001 controls mapping; GDPR data residency options; HIPAA audit support for healthcare tenants	Annual third-party audit; compliance documentation portal
Multi-Tenant Isolation	Complete data isolation between tenants at database, cache, and event stream layers	Tenant ID partitioned tables; Kafka topic-per-tenant option; namespace isolation in K8s

16. DevOps and Deployment Architecture

16.1 Containerization and Orchestration

- All REGIQ services are containerized using Docker with multi-stage builds to minimize image size
- Kubernetes (K8s) 1.28+ is the target orchestration platform; supports AWS EKS, Azure AKS, GCP GKE, and on-premise Kubernetes
- Helm charts provided for all services with configurable values per environment (dev, staging, prod)
- Horizontal Pod Autoscaling (HPA) configured for AI inference services based on GPU memory and CPU metrics
- Vertical Pod Autoscaler (VPA) for database and Kafka workloads

16.2 CI/CD Pipeline

Pipeline Stage	Tools	Gates / Quality Criteria
Source Control	GitHub Enterprise / GitLab	Branch protection; required reviews; signed commits
Build	GitHub Actions / Jenkins	Parallel builds per service; Docker multi-stage; build cache
Unit & Integration Tests	Jest (Node.js), pytest (Python), Supertest (API)	Coverage >80% gate; no P1/P2 test failures allowed
SAST & Dependency Scan	SonarQube, Snyk, Trivy	Block on critical/high severity vulnerabilities
Container Build & Push	Docker BuildKit; Amazon ECR / Azure ACR	Image tagged with git SHA; latest tag for main branch
Staging Deployment	Helm + ArgoCD (GitOps)	Automated deployment to staging on main branch merge
AI Model Validation	Custom pytest suite; MLflow model comparison	Model accuracy KPIs must meet threshold before promotion
Performance Testing	k6 / Locust against staging	P95 response time <500ms for API; <2s for impact analysis
Production Deployment	ArgoCD progressive delivery (canary/blue-green)	Canary at 5% → 25% → 100%; automated rollback on error rate spike
Monitoring Activation	Prometheus + Grafana + PagerDuty alerts	Alerts configured on error rate >1%, P99 latency >2s, CPU >80%

16.3 Observability Stack

- Metrics: Prometheus for service metrics; Grafana dashboards per service domain; custom REGIQ business metrics (analysis runs/hour, recommendation acceptance rate)
- Logging: Structured JSON logs from all services; Fluentd collection; Elasticsearch + Kibana (ELK) for log analysis; log retention 90 days
- Tracing: OpenTelemetry instrumentation across all services; Jaeger for distributed trace visualization; trace sampling at 5% in production, 100% in staging
- Alerting: PagerDuty integration for P1/P2 incidents; Slack channel integration for P3/P4; runbook links in all alerts

17. Platform Testing Strategy

Test Type	Scope	Tools	Pass Criteria
Unit Testing	Individual service functions, AI model inference functions, parser components	Jest, pytest, unittest	≥80% line coverage; zero critical test failures
Integration Testing	Service-to-service API contracts, ServiceNow integration, database operations	Supertest, pytest, Postman Newman	All API contracts validated; zero data integrity failures
AI Model Validation	Classification accuracy, impact analysis precision/recall, risk score calibration	Custom MLflow evaluation suite, scikit-learn metrics	See Section 4.2 AI Model KPIs
Performance Testing	API throughput, impact analysis latency, dependency graph query performance	k6, Locust	P95 API <500ms; Impact Analysis <60s for 1,000 components
Load Testing	Concurrent user simulation, Kafka throughput, database connection pool under load	k6 with ramping VUs, Kafka performance test	500 concurrent users; no degradation; zero data loss
Security Testing	OWASP Top 10, JWT validation, RBAC enforcement, SQL injection, XSS	OWASP ZAP, Burp Suite, manual pen test	Zero critical/high CVSS findings in production
UAT	End-to-end QA engineer workflows, release readiness scenarios, defect dashboard accuracy	Manual testing by enterprise QA pilot customers	≥90% user acceptance from pilot cohort
Graph Performance Testing	Neo4j query performance for large graphs (100K+ nodes)	Custom Neo4j query benchmark suite	Impact traversal <3s for 10-hop graph traversal on 100K nodes

18. Risk Register and Mitigation Strategies

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy
R-01	AI Impact Analysis produces false negatives — impacted test cases missed, leading to defect escapes	Medium	Critical	Ensemble model approach; mandatory inclusion of Critical/High tests for changed modules regardless of AI score; post-release retrospective feedback loop
R-02	AI misclassification of test cases — wrong business process or criticality assignment	Low-Medium	High	Human-in-loop review queue for low-confidence classifications; weekly accuracy audits; easy override workflow
R-03	Neo4j dependency graph performance degradation at enterprise scale (>1M nodes)	Medium	High	Graph partitioning by tenant and module; read replica cluster; query result caching in Redis; depth-limited traversal with index optimization
R-04	ServiceNow integration failure — sync breaks or update set parsing errors	Medium	High	Circuit breaker pattern; polling fallback for webhooks; sync health dashboard with alerting; manual re-sync capability
R-05	AI model drift — model accuracy degrades post-deployment due to evolving ServiceNow configurations	Medium	Medium	Monthly model evaluation against ground truth; automated retraining triggers on accuracy threshold breach; model versioning with rollback
R-06	Regulatory data residency requirements — test data may contain PII or confidential business logic	Low-Medium	High	Regional deployment option; tenant-level encryption keys; data classification engine; GDPR-compliant data export/delete API
R-07	User adoption resistance — QA teams distrust AI recommendations or revert to manual suites	Medium	High	Phased rollout with XAI explanations; side-by-side comparison (AI vs manual) in pilot; QA champion program; metrics showing AI accuracy vs baseline
R-08	Kafka event loss during high-volume change events (e.g., major ServiceNow upgrade)	Low	High	Kafka replication factor 3; idempotent producers; at-least-once delivery with deduplication in consumers; event replay capability
R-09	Incorrect confidence score calibration — overconfident AI leading to under-testing	Low-Medium	Critical	Platt scaling + isotonic regression for probability calibration; conservative default thresholds (min_confidence = 0.45); calibration metrics in model dashboard
R-10	Multi-tenant data leakage — tenant isolation breach in shared infrastructure	Very Low	Critical	Row-level security in PostgreSQL; Kafka topic isolation; namespace

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy
				isolation in K8s; annual penetration testing

19. Future Enhancements Roadmap

Enhancement	Description	Target Phase	Estimated Value
Autonomous Regression Selection	Fully autonomous suite generation and execution scheduling with no human approval required for standard releases	Phase 5	Additional 20% effort reduction beyond Phase 4
Generative AI Test Case Creation	GPT-4-based test case generation from user story text and ServiceNow component changes; auto-populated into repository with draft status	Phase 4	Reduce test authoring time by 40%
Self-Healing Test Suites	AI detects test case obsolescence from component changes and auto-updates steps; flags tests requiring human revision	Phase 5	Eliminate stale test maintenance effort
Predictive Release Quality Scoring	Pre-release defect probability forecast published 48 hours before deployment, with recommended risk mitigation actions	Phase 4	Improve go/no-go decision confidence
Conversational QA Assistant	LangChain-powered chatbot interface: 'What tests should I run for this change?', 'Why did this defect recur?', 'What is the blast radius of this update set?'	Phase 4	Reduce time to insight for QA engineers by 60%
AI Copilot for Test Authoring	Real-time AI suggestions during test case writing: auto-complete steps, suggest dependencies, warn of similar existing tests	Phase 3	Reduce authoring time by 30%
ServiceNow CSM/FSM Extension	Extend test classification and dependency mapping to Customer Service Management and Field Service Management modules	Phase 3	Expand addressable market
Automated Defect Filing	AI generates and files defect tickets in Jira/ADO when test execution fails, pre-populated with component analysis and impact context	Phase 4	Eliminate manual defect documentation effort
Cross-Tenant Benchmark Analytics	Anonymized aggregate benchmarking: compare defect leakage rates, AI accuracy, and coverage against industry peers	Phase 5	Deliver industry intelligence value

20. Success Metrics Framework

Metric	Baseline (Pre-REGIQ)	6-Month Target	12-Month Target	Measurement Method
Regression Effort (hrs/release)	Organization-specific baseline	-35%	-55%	Time tracking in project management tool
Production Defect Leakage Rate	Baseline defects/release	-25%	-45%	Post-release incident count analysis
Impact Analysis Accuracy (Precision)	N/A (manual — no baseline)	>85%	>92%	Retrospective labeling of true impacted tests
AI Classification Accuracy	N/A	>88%	>93%	Monthly classification audit vs manual label
Release Cycle Duration (days)	Org-specific baseline	-20%	-35%	Release management system metrics
Mean Time to Validation (MTTV)	Org-specific baseline (days)	-40%	-60%	Change detection to regression complete timestamp
Repository Coverage (% mapped)	Typically 30–50%	>80%	>95%	Dashboard: unmapped test case count
Risk Score Correlation (Spearman ρ)	N/A	>0.75	>0.85	Post-release comparison vs predicted risk
User Adoption Rate	0%	>70% active weekly users	>90% active weekly users	Platform usage analytics
AI Recommendation Acceptance Rate	N/A	>65%	>80%	Accept/reject events logged in platform

21. Product Roadmap

Phase	Name	Duration	Key Deliverables	Exit Criteria
Phase 1	Repository Platform	Q3 2025 (3 months)	Enterprise test repository, ServiceNow integration, basic classification, Excel import/export, user management, RBAC	500+ test cases imported; sync operational; all RBAC roles functional
Phase 2	Intelligent Classification	Q4 2025 (3 months)	AI classification engine (TCC model), criticality scoring, dependency mapping UI, defect ingestion, basic dashboards	AI classification accuracy >88%; dependency graph operational for 1 module
Phase 3	AI-Based Impact Analysis	Q1–Q2 2026 (4 months)	Impact Analysis Engine (IAR, SEM, DGR), Neo4j integration, confidence scoring, XAI panels, CI/CD API integration, regression suite engine	Impact analysis precision >85%; CI/CD pipeline integration live; 3 pilot customers
Phase 4	Predictive Intelligence	Q3–Q4 2026 (4 months)	Defect Recurrence Predictor, Risk Score Model, Release Readiness Score, predictive defect forecasting, AI copilot (beta), generative test case suggestions	Risk score correlation >0.82; 10 enterprise customers; AI copilot beta feedback
Phase 5	Autonomous QA Platform	2027 (ongoing)	Autonomous suite selection, self-healing tests, cross-tenant benchmarking, conversational QA assistant (GA), full autonomous release validation option	Autonomous mode adopted by >30% of customers; MTTV <24 hours median

22. Project Deliverables

Deliverable	Description	Phase	Acceptance Criteria
Web Platform (SaaS)	Full-stack Next.js + Node.js application with all modules described in Section 8	Phase 1–3	All FR requirements validated in UAT; performance benchmarks met
AI Services Suite	Python FastAPI AI services: TCC, IAR, SEM, DRP, RSM, DGR, NCA models deployed and operational	Phase 2–3	AI KPIs from Section 4.2 achieved on pilot customer data
REST API Layer	Complete documented API layer per Section 13 with OpenAPI 3.1 specification and Postman collection	Phase 1	All endpoints operational; 100% documented in OpenAPI spec
Dependency Graph Engine	Neo4j-backed dependency mapping engine with React-Flow visualization	Phase 2–3	Graph traversal accuracy >95% vs manual mapping on test ServiceNow instance
ServiceNow Integration Package	OAuth2 connector, update set parser, incremental sync engine, webhook listener	Phase 1	Successful sync with DEV/TEST/UAT/PROD instances; update set parsing >95% accuracy
Reporting & Dashboard Suite	All dashboards from Section 8.8 with export capability (PDF, Excel)	Phase 1–3	All stakeholder dashboards approved in UAT
CI/CD Integration SDK	NPM package and Python library for integrating REGIQ into Jenkins/GitHub Actions pipelines	Phase 3	Reference CI/CD integration working in at least 2 major pipeline tools
Deployment Package	Docker Compose (dev), Helm charts (K8s), Terraform IaC for AWS/Azure, deployment runbooks	Phase 1	Deployment completed in <2 hours from scratch using provided documentation
Technical Documentation	Architecture guide, API reference, integration guide, admin guide, user guide, AI model card	Ongoing	Documentation completeness >95% per checklist; pilot customer sign-off
AI Model Cards	Transparency documentation per model: training data, performance metrics, limitations, bias analysis	Phase 2–3	Model card completed for all 7 AI models; reviewed by AI ethics reviewer

— END OF DOCUMENT —

REGIQ — AI-Powered ServiceNow ITSM Regression Test Suite Management & Intelligent Impact Analysis
Platform
Product Requirements Document v1.0 | Confidential | © 2025 REGIQ Platform Division